JavaScript Asynchronous Programming Activities

Code + Console/Preview Output per Activity

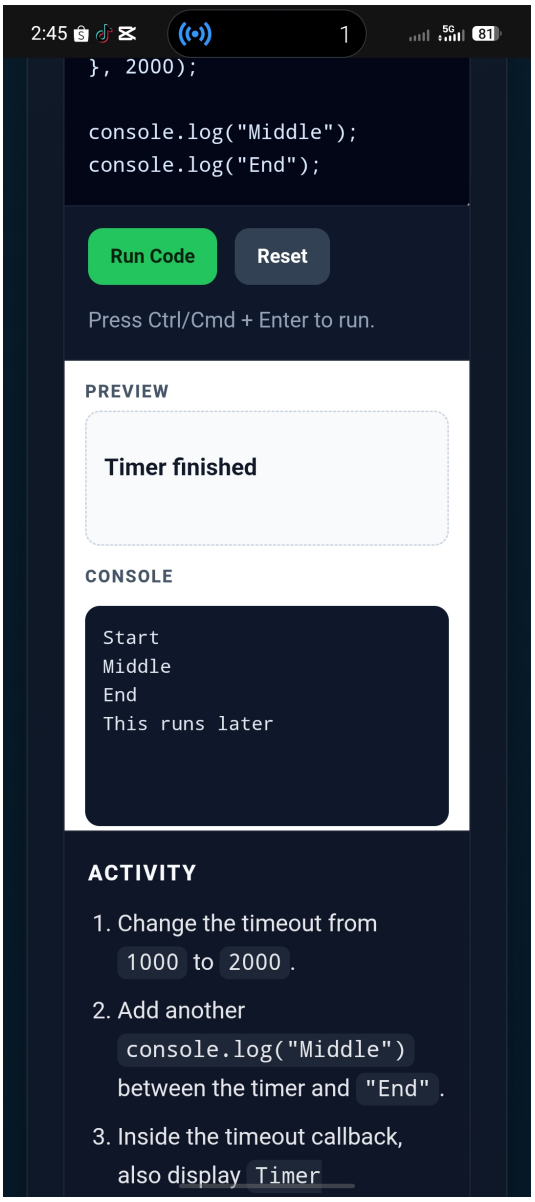
1. Synchronous vs Asynchronous

```
console.log("Start");

setTimeout(() => {
  console.log("This runs later");
  app.innerHTML = "<p><strong>Timer finished</strong></p>";
}, 2000);

console.log("Middle");
console.log("End");
```

Output:

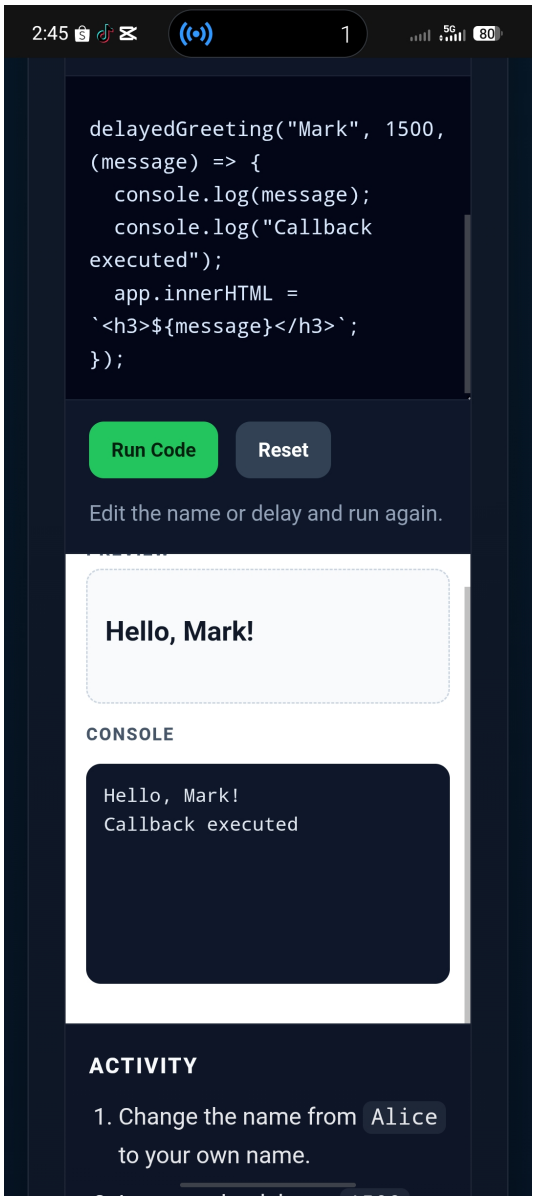


2. Simple Callback

```
function delayedGreeting(name, ms, callback) {
  setTimeout(() => {
    callback(`Hello, ${name}!`);
  }, ms);
}

delayedGreeting("Mark", 1500, (message) => {
  console.log(message);
  console.log("Callback executed");
  app.innerHTML = `
    <h3>${message}</h3>
    <p>Callback executed</p>
  `;
});
```

Output:



3. Callback Hell

```
function getUser(id, callback) {
  setTimeout(() => callback({ id, name: "Maria" }), 400);
}

function getOrders(userId, callback) {
  setTimeout(() => callback([{ id: 101, userId }]), 400);
}

function getShippingStatus(orderId, callback) {
  setTimeout(() => callback("Delivered"), 400);
}

function getDeliveryDate(orderId, callback) {
  setTimeout(() => callback("September 4, 2026"), 400);
}

getUser(1, (user) => {
  console.log("User:", user.name);

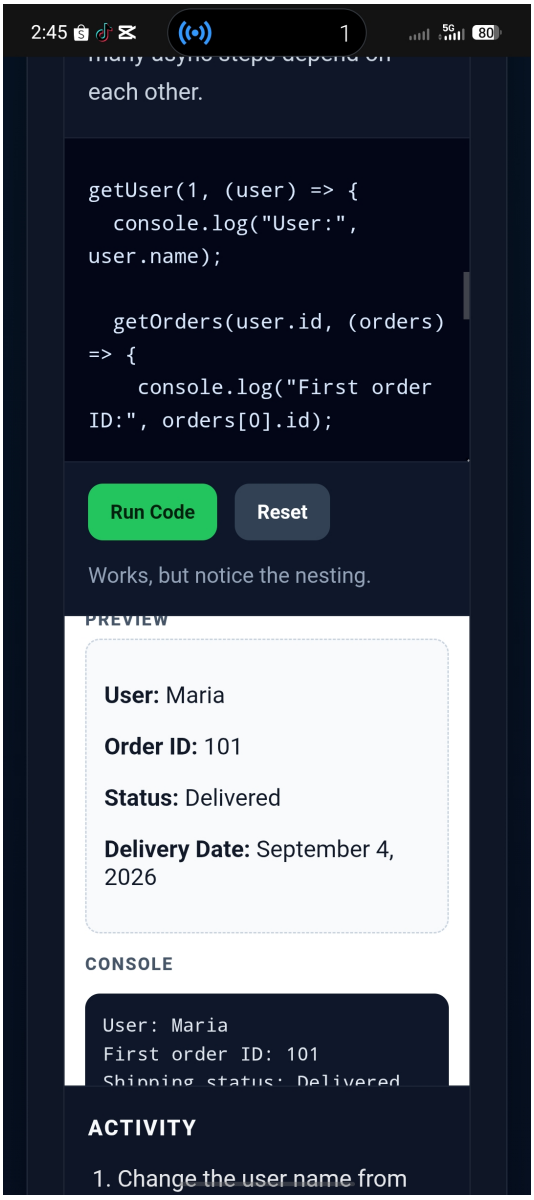
  getOrders(user.id, (orders) => {
    console.log("First order ID:", orders[0].id);

    getShippingStatus(orders[0].id, (status) => {
      console.log("Shipping status:", status);

      getDeliveryDate(orders[0].id, (date) => {
        console.log("Delivery date:", date);

        app.innerHTML = `
          <p><strong>User:</strong> ${user.name}</p>
          <p><strong>Order ID:</strong> ${orders[0].id}</p>
          <p><strong>Status:</strong> ${status}</p>
          <p><strong>Delivery Date:</strong> ${date}</p>
        `;
      });
    });
  });
});
```

Output:


2:45
each other.
getUser(1, (user) => {
console.log("User:", user.name);
getOrders(user.id, (orders) => {
console.log("First order ID:", orders[0].id);
Run Code
Reset
Works, but notice the nesting.
PREVIEW
User: Maria
Order ID: 101
Status: Delivered
Delivery Date: September 4, 2026
CONSOLE
User: Maria
First order ID: 101
ACTIVITY
1. Change the user name from

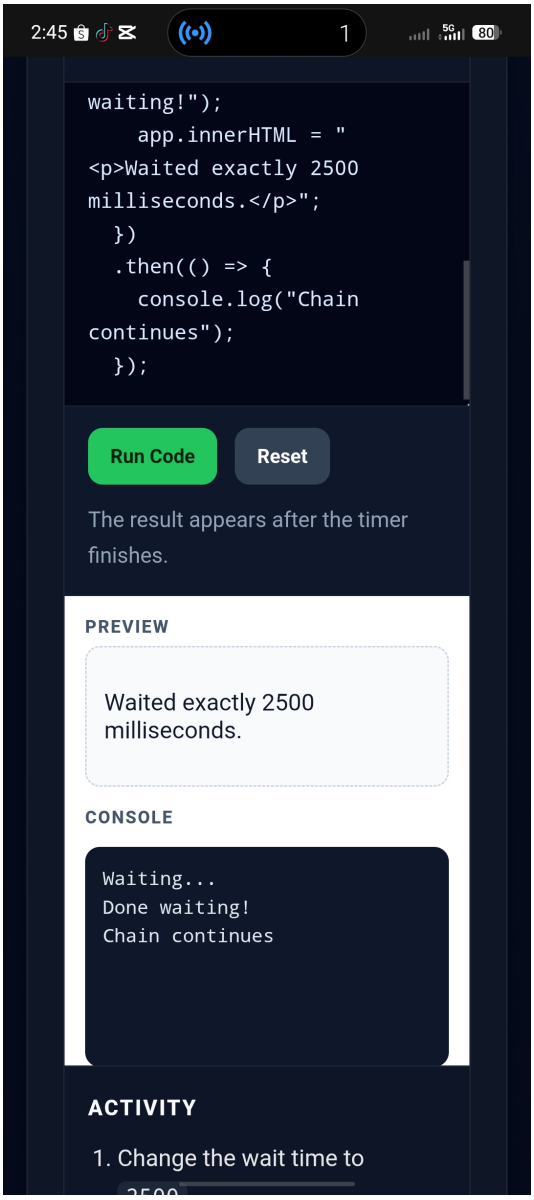
4. Promise — Create and Use

```
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

console.log("Waiting...");

wait(2500)
  .then(() => {
    console.log("Done waiting!");
    app.innerHTML = "<p>Waited exactly 2500 milliseconds.</p>";
  })
  .then(() => {
    console.log("Chain continues");
    app.innerHTML += "<p>Chain continues</p>";
  });
```

Output:


2:45
1
80
waiting!");
app.innerHTML = "
Waited exactly 2500
milliseconds. ";
})
.then(() => {
console.log("Chain
continues");
});
Run Code
Reset
The result appears after the timer finishes.
PREVIEW
Waited exactly 2500 milliseconds.
CONSOLE
Waiting...
Done waiting!
Chain continues
ACTIVITY
1. Change the wait time to

5. Promise Chaining

```
function getUser(id) {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ id, name: "Maria" }), 400);
  });
}

function getOrders(userId) {
  return new Promise((resolve) => {
    setTimeout(() => resolve([{ id: 101, userId }]), 400);
  });
}

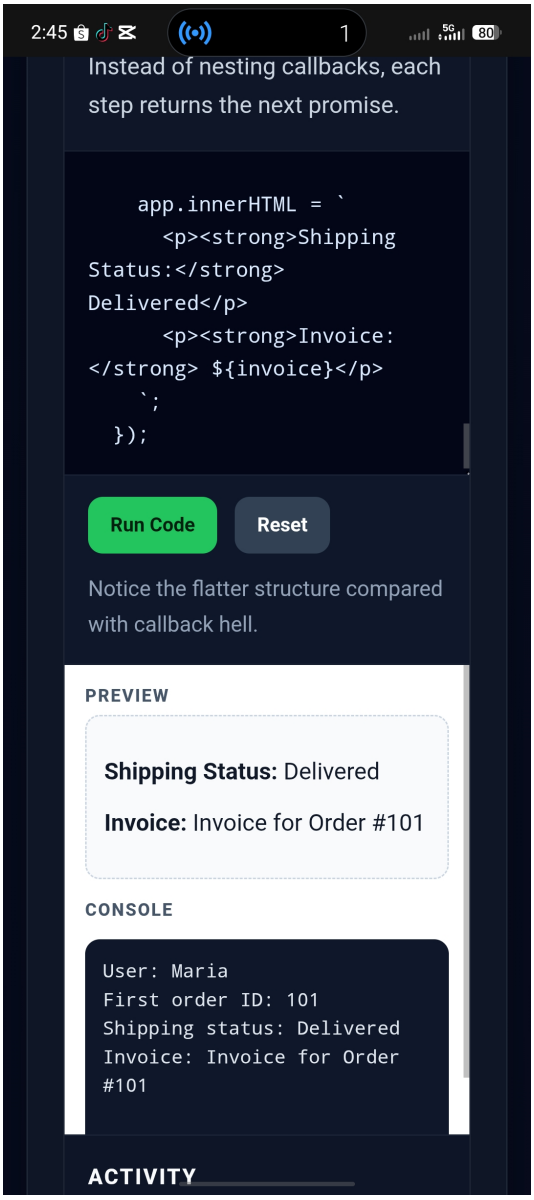
function getShippingStatus(orderId) {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Delivered"), 400);
  });
}

function getInvoice(orderId) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`Invoice for Order #${orderId}`), 400);
  });
}

getUser(1)
  .then((user) => {
    console.log("User:", user.name);
    return getOrders(user.id);
  })
  .then((orders) => {
    console.log("First order ID:", orders[0].id);
    return getShippingStatus(orders[0].id);
  })
  .then((status) => {
    console.log("Shipping status:", status);
    return getInvoice(101);
  })
  .then((invoice) => {
    console.log("Invoice:", invoice);

    app.innerHTML = `
      <p><strong>User:</strong> Maria</p>
      <p><strong>Order ID:</strong> 101</p>
      <p><strong>Shipping Status:</strong> Delivered</p>
      <p><strong>Invoice:</strong> ${invoice}</p>
    `;
  });
```

Output:


2:45
Instead of nesting callbacks, each step returns the next promise.
app.innerHTML = `
Shipping Status: Delivered
Invoice: ${invoice}
`;
});
Run Code
Reset
Notice the flatter structure compared with callback hell.
PREVIEW
Shipping Status: Delivered
Invoice: Invoice for Order #101
CONSOLE
User: Maria
First order ID: 101
Shipping status: Delivered
Invoice: Invoice for Order #101
ACTIVITY

6. Basic Async/Await

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function main() {
  console.log("Before await");

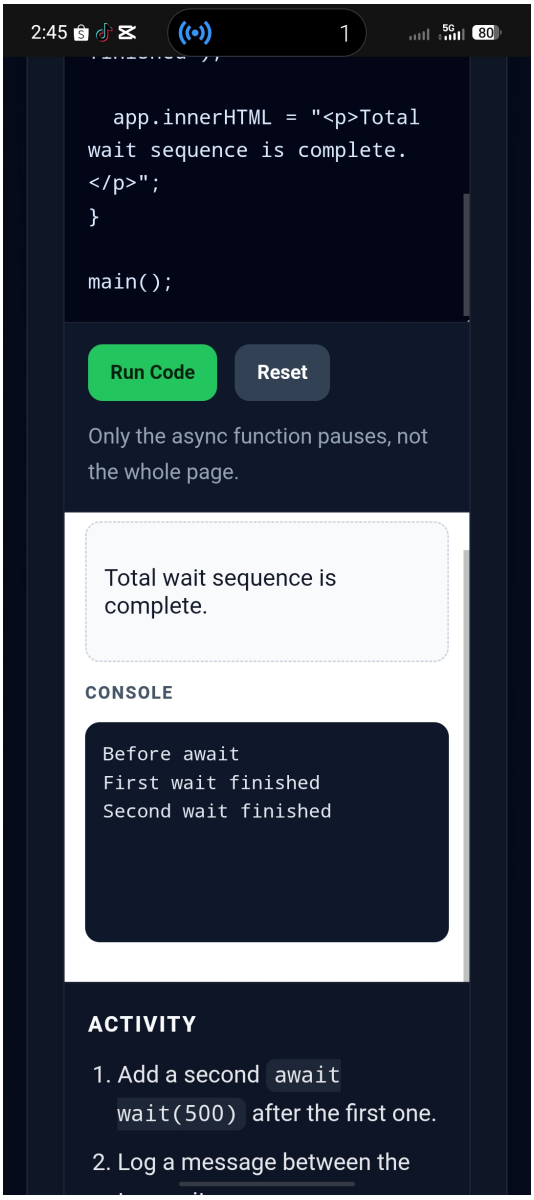
  await wait(1000);
  console.log("First wait finished");

  await wait(500);
  console.log("Second wait finished");

  app.innerHTML = `
    <p><strong>Before await</strong></p>
    <p>First wait finished</p>
    <p>Second wait finished</p>
    <p><strong>Total wait sequence is complete.</strong></p>
  `;
}

main();
```

Output:


2:45
1
app.innerHTML = " Total wait sequence is complete. ";
}
main();
Run Code
Reset
Only the async function pauses, not the whole page.
Total wait sequence is complete.
CONSOLE
Before await
First wait finished
Second wait finished
ACTIVITY
1. Add a second await wait(500) after the first one.
2. Log a message between the

7. Async/Await with try/catch

```
function riskyOperation(shouldFail) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldFail) {
        reject(new Error("Something went wrong"));
      } else {
        resolve("Everything worked");
      }
    }, 700);
  });
}

async function runDemo() {
  try {
    const result = await riskyOperation(true);

    console.log(result);
    app.innerHTML = `<p>${result}</p>`;

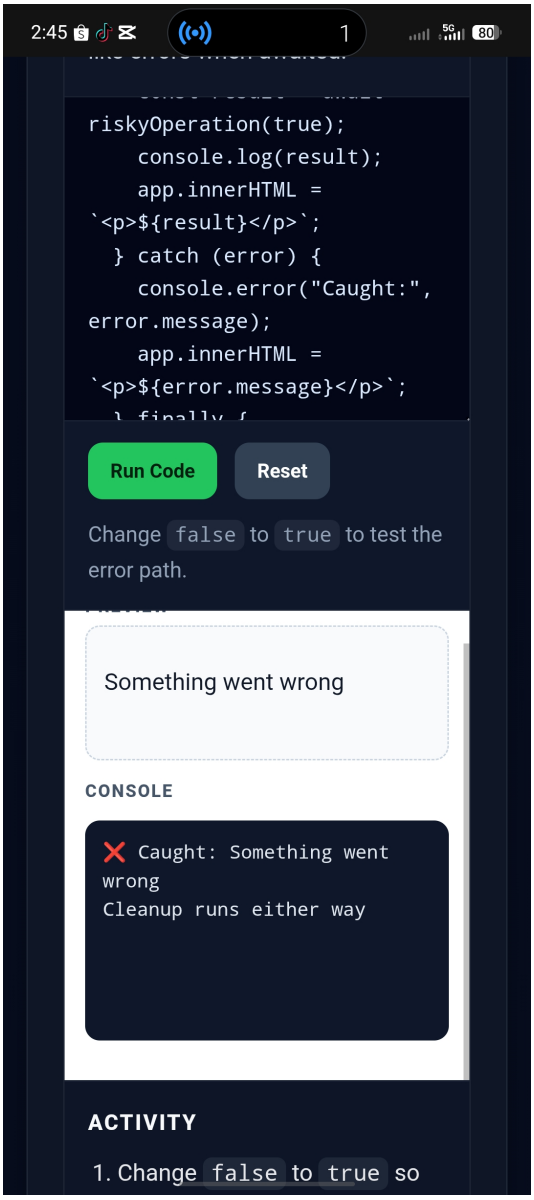
  } catch (error) {
    console.error("Caught:", error.message);

    app.innerHTML = `
      <p><strong>Error:</strong> ${error.message}</p>
      <p>The promise was rejected.</p>
    `;

  } finally {
    console.log("Cleanup runs either way");
  }
}

runDemo();
```

Output:


2:45
1
riskyOperation(true);
console.log(result);
app.innerHTML =
`<p>${result}</p>`;
} catch (error) {
console.error("Caught:",
error.message);
app.innerHTML =
`<p>${error.message}</p>`;
Run Code
Reset
Change false to true to test the error path.
Something went wrong
CONSOLE
❌ Caught: Something went wrong
Cleanup runs either way
ACTIVITY
1. Change false to true so

8. Promise.all()

```
function task(name, ms) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log(`${name} finished`);
      resolve(name);
    }, ms);
  });
}

const start = Date.now();

const results = await Promise.all([
  task("Task A", 1000),
  task("Task B", 1500),
  task("Task C", 700),
  task("Task D", 300)
]);

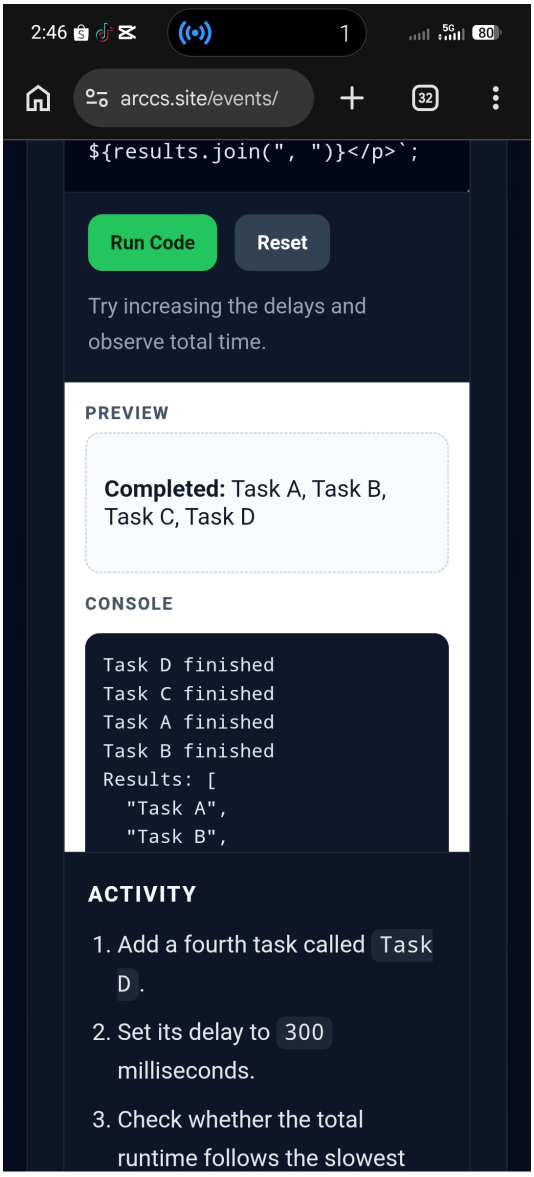
const totalTime = Date.now() - start;

console.log("Results:", results);
console.log("Total time:", `${totalTime}ms`);

app.innerHTML = `
  <p><strong>Completed:</strong> ${results.join(", ")}</p>
  <p><strong>Total time:</strong> ${totalTime}ms</p>
  <p>Tasks ran in parallel.</p>
`;
```

Important: Total time should be around 1500 ms, not 3500 ms.

Output:


2:46
arccs.site/events/
${results.join(", ")}</p>`;
Run Code
Reset
Try increasing the delays and observe total time.
PREVIEW
Completed: Task A, Task B, Task C, Task D
CONSOLE
Task D finished
Task C finished
Task A finished
Task B finished
Results: [
"Task A",
"Task B",
ACTIVITY
1. Add a fourth task called Task D.
2. Set its delay to 300 milliseconds.
3. Check whether the total runtime follows the slowest

9. allSettled(), any(), race()

```
function succeed(name, ms) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`${name} success`), ms);
  });
}

function fail(name, ms) {
  return new Promise((_, reject) => {
    setTimeout(() => reject(new Error(`${name} failed`)), ms);
  });
}

const settled = await Promise.allSettled([
  succeed("A", 400),
  fail("B", 700),
  succeed("C", 250),
  fail("D", 500)
]);

console.log("allSettled result:", settled);

const firstFulfilled = await Promise.any([
  fail("X", 300),
  succeed("Y", 800),
  succeed("Z", 200)
]);

console.log("Promise.any winner:", firstFulfilled);

let raceResult;

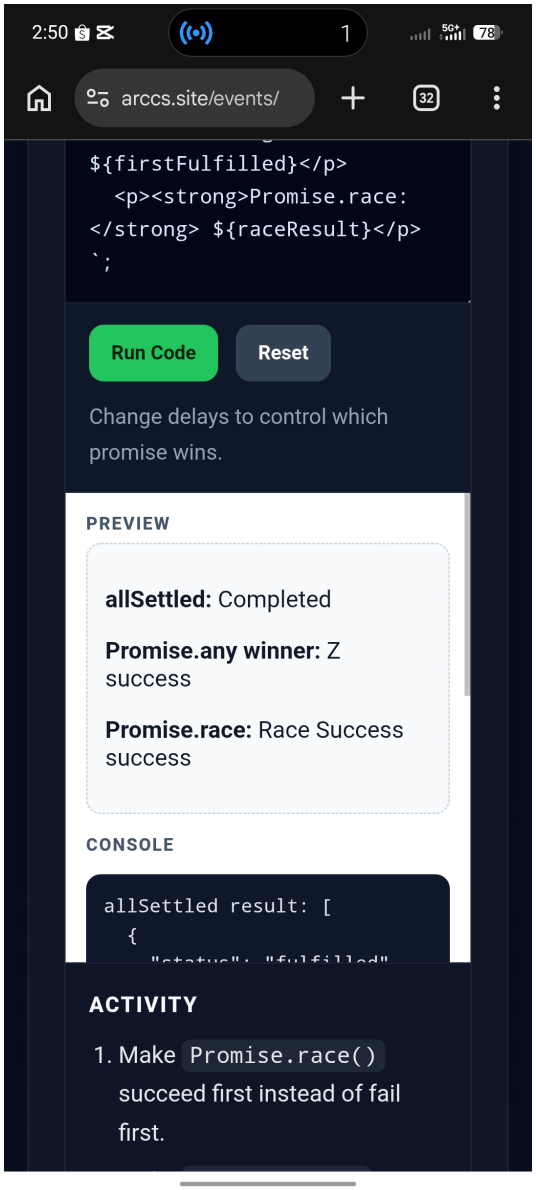
try {
  raceResult = await Promise.race([
    succeed("Race Success", 200),
    fail("Race Error", 350)
  ]);

  console.log("Promise.race result:", raceResult);

} catch (error) {
  console.error("Promise.race rejected first:", error.message);
}

app.innerHTML = `
  <p><strong>allSettled:</strong> Completed</p>
  <p><strong>Promise.any winner:</strong> ${firstFulfilled}</p>
  <p><strong>Promise.race:</strong> ${raceResult}</p>
`;
```

Output:


2:50
arccs.site/events/
32
${firstFulfilled}
Promise.race:
 ${raceResult}
`;
Run Code
Reset
Change delays to control which promise wins.
PREVIEW
allSettled: Completed
Promise.any winner: Z success
Promise.race: Race Success success
CONSOLE
allSettled result: [
{
ACTIVITY
1. Make Promise.race() succeed first instead of fail first.

10. Event Loop

```
console.log("1: sync");

setTimeout(() => {
  console.log("2: macrotask");
}, 0);

Promise.resolve().then(() => {
  console.log("3: microtask");
});

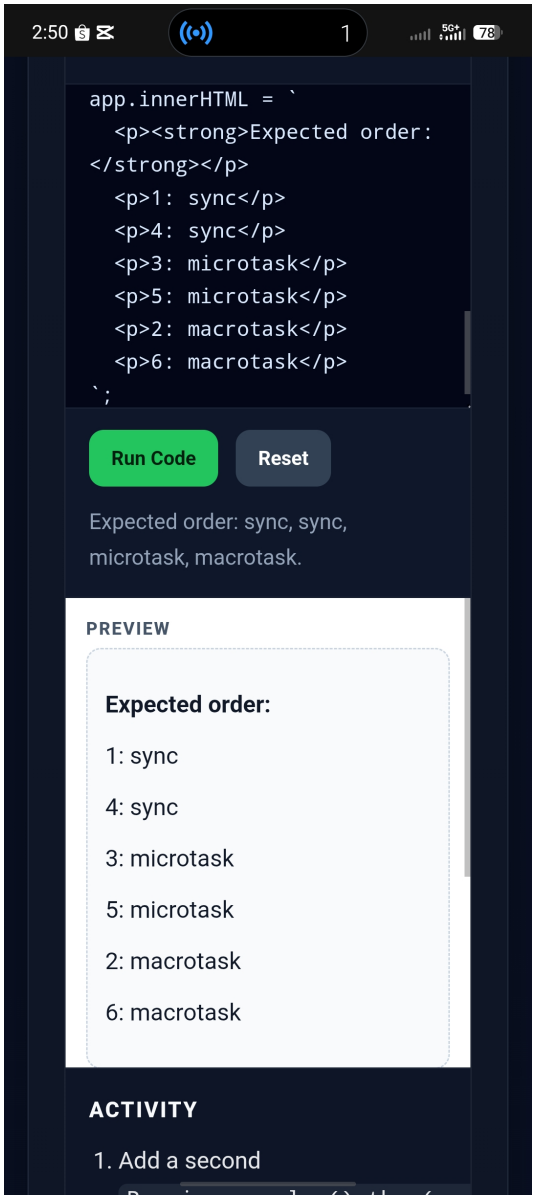
console.log("4: sync");

Promise.resolve().then(() => {
  console.log("5: microtask");
});

setTimeout(() => {
  console.log("6: macrotask");
}, 0);

app.innerHTML = `
  <p><strong>Expected order:</strong></p>
  <p>1: sync</p>
  <p>4: sync</p>
  <p>3: microtask</p>
  <p>5: microtask</p>
  <p>2: macrotask</p>
  <p>6: macrotask</p>
`;
```

Output:


2:50
1
app.innerHTML = `
Expected order:

1: sync
4: sync
3: microtask
5: microtask
2: macrotask
6: macrotask
`;
Run Code
Reset
Expected order: sync, sync, microtask, macrotask.
PREVIEW
Expected order:
1: sync
4: sync
3: microtask
5: microtask
2: macrotask
6: macrotask
ACTIVITY
1. Add a second

11. Fetch API

```
console.log("Loading user...");

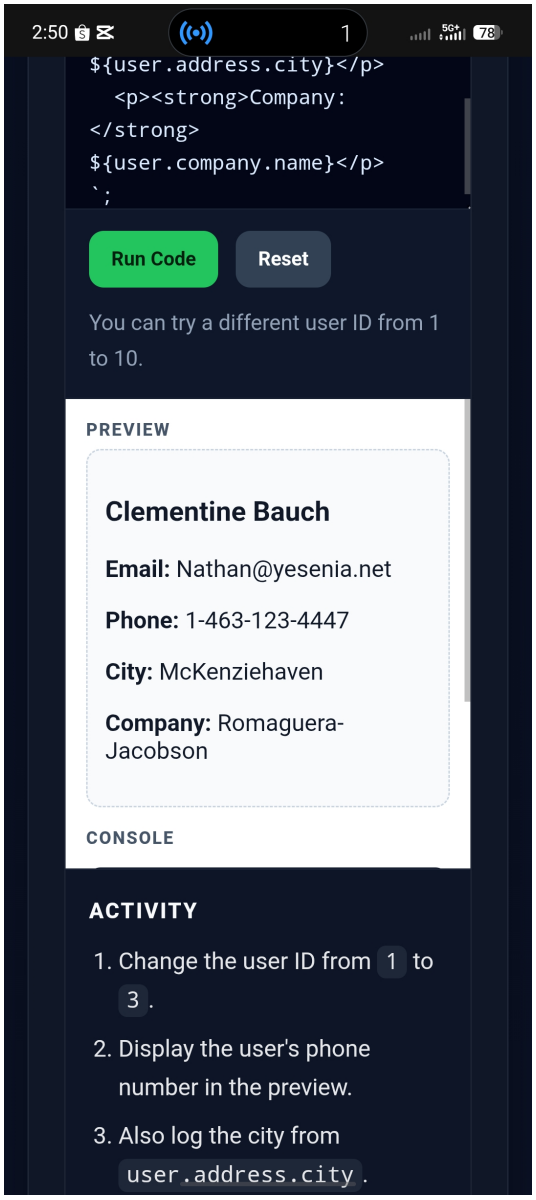
const response = await fetch(
  "https://jsonplaceholder.typicode.com/users/3"
);

const user = await response.json();

console.log("User name:", user.name);
console.log("Email:", user.email);
console.log("City:", user.address.city);

app.innerHTML = `
  <h3>${user.name}</h3>
  <p><strong>Email:</strong> ${user.email}</p>
  <p><strong>Phone:</strong> ${user.phone}</p>
  <p><strong>City:</strong> ${user.address.city}</p>
  <p><strong>Company:</strong> ${user.company.name}</p>
`;
```

Output:


2:50
1
${user.address.city}
Company:

${user.company.name}
`;
Run Code
Reset
You can try a different user ID from 1 to 10.
PREVIEW
Clementine Bauch
Email: Nathan@yesenia.net
Phone: 1-463-123-4447
City: McKenziehaven
Company: Romaguera-Jacobson
CONSOLE
ACTIVITY
1. Change the user ID from 1 to 3.
2. Display the user's phone number in the preview.
3. Also log the city from user.address.city.

12. Fetch response.ok

```
try {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts/1"
  );

  if (!response.ok) {
    throw new Error(`HTTP error! Status: ${response.status}`);
  }

  const data = await response.json();

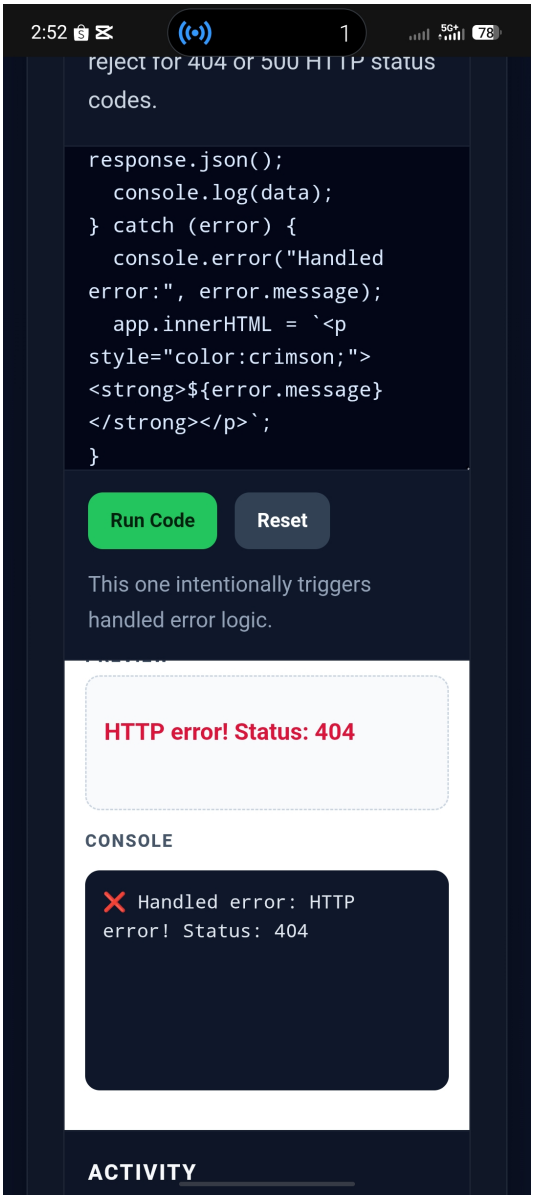
  console.log(data);

  app.innerHTML = `
    <h3>${data.title}</h3>
    <p><strong>Post ID:</strong> ${data.id}</p>
  `;

} catch (error) {
  console.error("Handled error:", error.message);

  app.innerHTML = `
    <p><strong>Error:</strong> ${error.message}</p>
  `;
}
```

Output:


2:52
reject for 404 or 500 HTTP status codes.
response.json();
console.log(data);
} catch (error) {
console.error("Handled error:", error.message);
app.innerHTML = `<p style="color:crimson;"><strong>${error.message}</strong></p>`;
}
Run Code
Reset
This one intentionally triggers handled error logic.
HTTP error! Status: 404
CONSOLE
❌ Handled error: HTTP error! Status: 404
ACTIVITY

13. Forgetting await

```
async function getUser() {
  return {
    name: "Alice",
    role: "Admin",
    department: "IT"
  };
}

const userPromise = getUser();

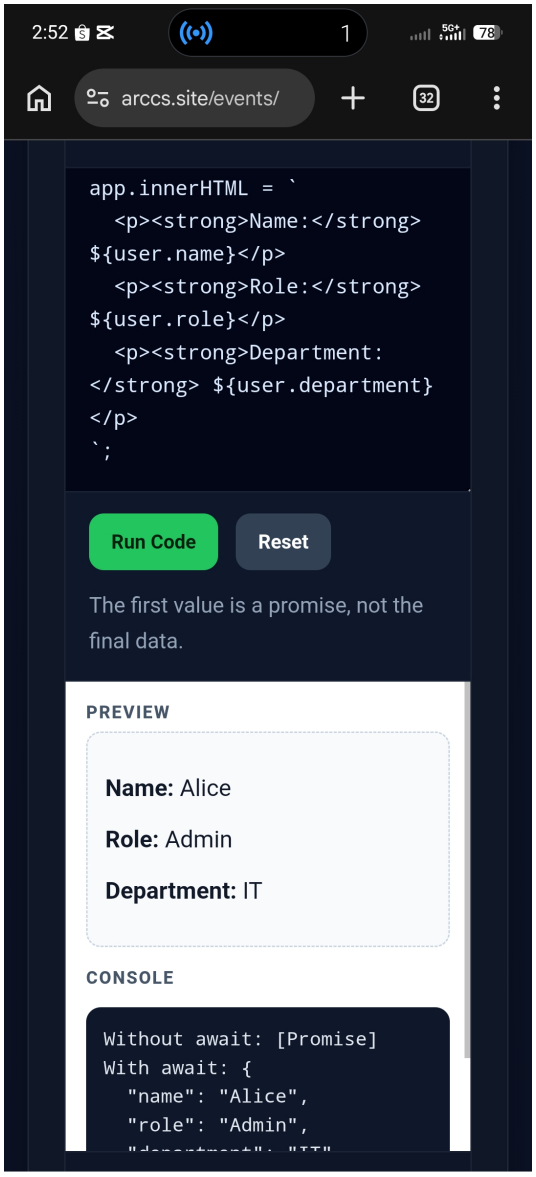
console.log("Without await:", userPromise);

const user = await getUser();

console.log("With await:", user);

app.innerHTML = `
  <p><strong>Name:</strong> ${user.name}</p>
  <p><strong>Role:</strong> ${user.role}</p>
  <p><strong>Department:</strong> ${user.department}</p>
`;
```

Output:


2:52
arccs.site/events/
app.innerHTML = `
Name:
${user.name}
Role:
${user.role}
Department:
 ${user.department}

`;
Run Code
Reset
The first value is a promise, not the final data.
PREVIEW
Name: Alice
Role: Admin
Department: IT
CONSOLE
Without await: [Promise]
With await: {
"name": "Alice",
"role": "Admin",

14. forEach(async...) Pitfall

```
function wait(ms, label) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log(label);
      resolve();
    }, ms);
  });
}

console.log("Using forEach...");

[1, 2, 3, 4].forEach(async (n) => {
  await wait(n * 400, `Finished item ${n}`);
});

console.log("This prints before the async work is done.");

app.innerHTML = `
  <p><strong>Using forEach...</strong></p>
  <p>This prints before the async work is done.</p>
  <p>forEach does not wait for async callbacks.</p>
`;
```

Output:


2:53
arccs.site/events/
Run Code
Reset
Try rewriting with for...of or Promise.all().
PREVIEW
Using forEach...
This prints before the async work is done.
forEach does not wait for async callbacks.
CONSOLE
Using forEach...
This prints before the async work is done.
ACTIVITY
1. Change the array to [1, 2, 3, 4].
2. Observe that the line after forEach still runs first.
3. Write a short note for yourself: why does forEach not wait?

15. Sequential for...of

```
function wait(ms, label) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log(label);
      resolve();
    }, ms);
  });
}

console.log("Sequential loop starts");

app.innerHTML = "<p><strong>Sequential loop starts</strong></p>";

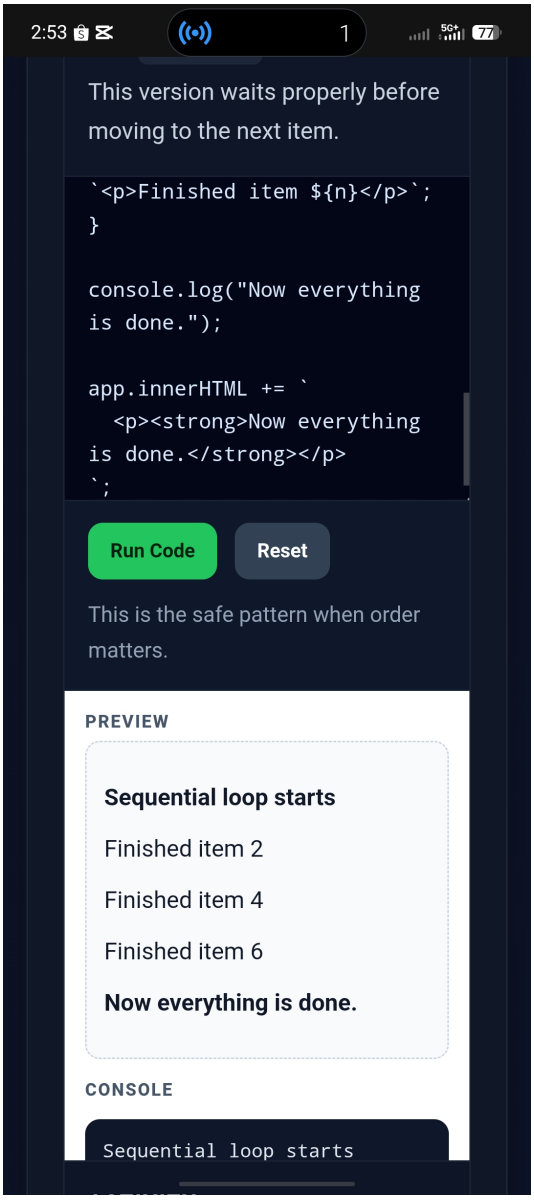
for (const n of [2, 4, 6]) {
  await wait(n * 300, `Finished item ${n}`);

  app.innerHTML += `<p>Finished item ${n}</p>`;
}

console.log("Now everything is done.");

app.innerHTML += `
  <p><strong>Now everything is done.</strong></p>
`;
```

Output:


2:53
This version waits properly before moving to the next item.
`<p>Finished item ${n}</p>`;
}
console.log("Now everything is done.");
app.innerHTML += `
Now everything is done.
`;
Run Code
Reset
This is the safe pattern when order matters.
PREVIEW
Sequential loop starts
Finished item 2
Finished item 4
Finished item 6
Now everything is done.
CONSOLE
Sequential loop starts

16. Debugging Async Code

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

console.log("[1] Start:", new Date().toLocaleTimeString());

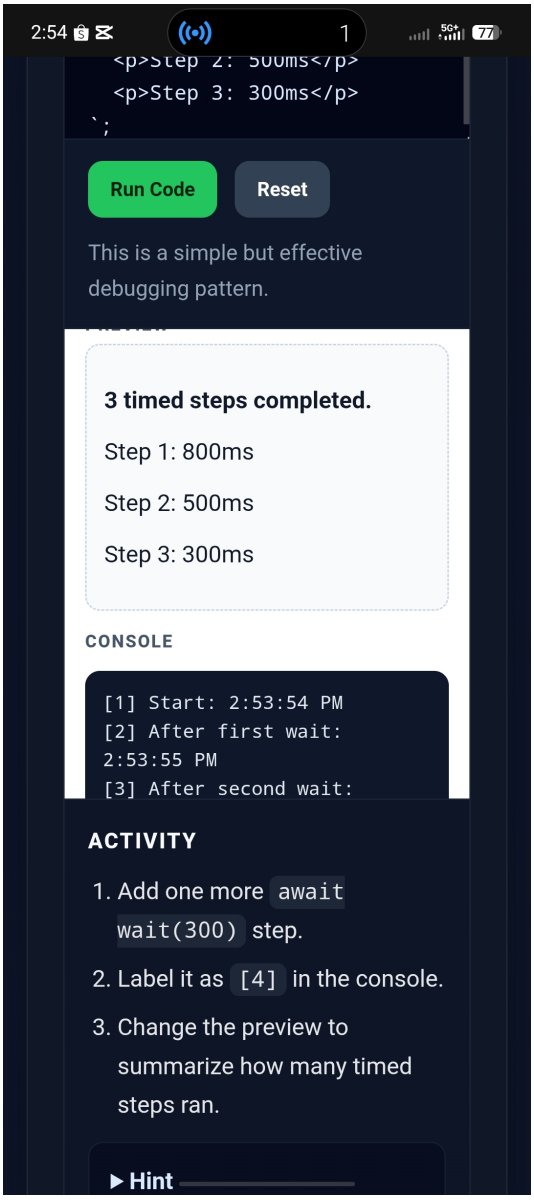
await wait(800);
console.log("[2] After first wait:", new Date().toLocaleTimeString());

await wait(500);
console.log("[3] After second wait:", new Date().toLocaleTimeString());

await wait(300);
console.log("[4] After third wait:", new Date().toLocaleTimeString());

app.innerHTML = `
  <p><strong>3 timed steps completed.</strong></p>
  <p>Step 1: 800ms</p>
  <p>Step 2: 500ms</p>
  <p>Step 3: 300ms</p>
`;
```

Output:


2:54
Step 2: 500ms
Step 3: 300ms
`;
Run Code
Reset
This is a simple but effective debugging pattern.
3 timed steps completed.
Step 1: 800ms
Step 2: 500ms
Step 3: 300ms
CONSOLE
[1] Start: 2:53:54 PM
[2] After first wait: 2:53:55 PM
[3] After second wait:
ACTIVITY
1. Add one more await wait(300) step.
2. Label it as [4] in the console.
3. Change the preview to summarize how many timed steps ran.
▶ Hint

17. AbortController

```
function fakeFetch({ signal }) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      resolve({
        message: "Loaded successfully after 5 seconds"
      });
    }, 5000);

    signal.addEventListener("abort", () => {
      clearTimeout(timer);
      reject(new DOMException("Aborted", "AbortError"));
    });
  });
}

const controller = new AbortController();

setTimeout(() => {
  controller.abort();
}, 6000);

try {
  console.log("Starting fake request...");

  const result = await fakeFetch({
    signal: controller.signal
  });

  console.log(result.message);
  console.log("Request completed.");

  app.innerHTML = `
    <p><strong>Request completed.</strong></p>
    <p>${result.message}</p>
  `;

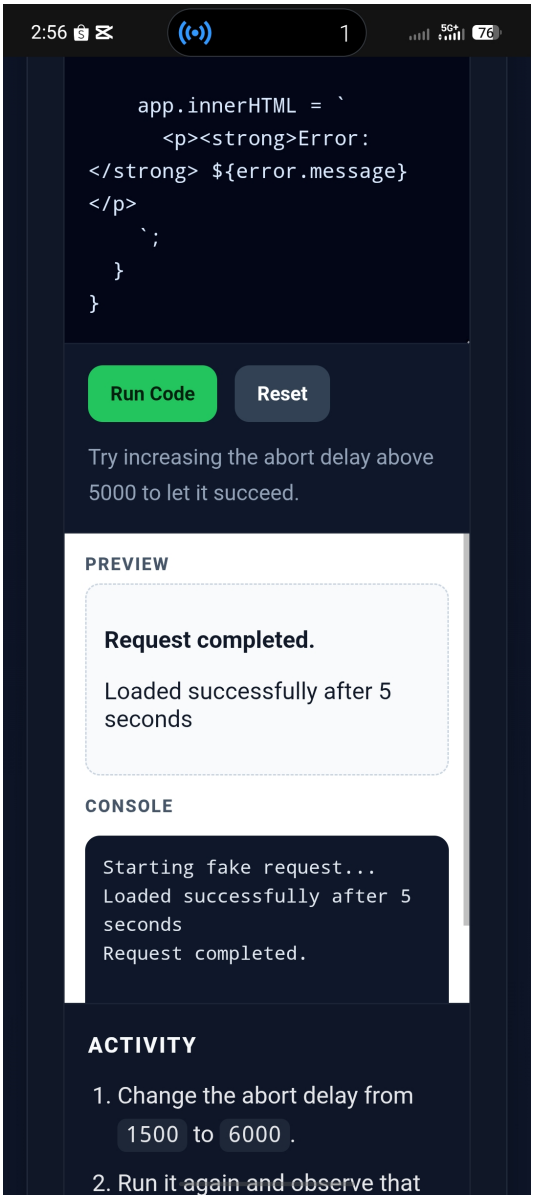
} catch (error) {
  if (error.name === "AbortError") {
    console.error("Request was aborted.");

    app.innerHTML = `
      <p><strong>Request was aborted.</strong></p>
    `;
  } else {
    console.error(error);

    app.innerHTML = `
      <p><strong>Error:</strong> ${error.message}</p>
    `;
  }
}
```

Important: Sa #17, 6000ms ang abort at 5000ms ang request, kaya "completed successfully" ang dapat makita sa Preview.

Output:


2:56
1
app.innerHTML = `
Error:
 ${error.message}

`;
}
}
Run Code
Reset
Try increasing the abort delay above 5000 to let it succeed.
PREVIEW
Request completed.
Loaded successfully after 5 seconds
CONSOLE
Starting fake request...
Loaded successfully after 5 seconds
Request completed.
ACTIVITY
1. Change the abort delay from 1500 to 6000.
2. Run it again and observe that